

Length of the longest subarray with zero sum

Given an array containing both +ve and -ve integers, we have to find the length of the longest subarray with the sum of all elements equal to zero.

I/p: $N=6$, array = $\{9, -3, 3, -1, 6, -5\}$

O/p: $\{-3, 3\} \Rightarrow 2$
 $\{-1, 6, -5\} \Rightarrow 3$
 $\{-3, 3, -1, 6, -5\} \Rightarrow 5$

I/p: $N=8$, array = $\{6, -2, 2, -8, 1, 7, 4, -10\}$

O/p: $\{-2, 2\} = 2$
 $\{-8, 1, 7\} = 3$
 $\{-2, 2, -8, 1, 7\} = 5$
 $\{6, -2, 2, -8, 1, 7, 4, -10\} = 8$

Brute force

1. Initialize variable $max = 0$

2. for $i \rightarrow 0$ to $n-1$

sum = 0

for $j \rightarrow i$ to $n-1$

sum += arr[j]

if $sum == 0$

$max = \max(max, j - i + 1)$
length of subarray

3. return max

Tc: $O(N^2)$

Sc: $O(1)$

Optimized Approach

1. $sum = 0, max = 0$

2. Create a map to store the prefix sum of every element as a key and its index as a value.

Case 1: subarray has $sum == 0$
 $max = \max(max, i + 1)$

Case 2: prefixsum seen before
 $max = \max(max, i - previndex)$

Case 3: prefixsum seeing for first time
put (prefixsum, i) in map

arr:	⁰ 15	¹ -2	² 2	³ -8	⁴ 1	⁵ 7	⁶ 10	⁷ 23
sum = $\emptyset$	15	13	15	7	8	15	25	48
sum = 0:	X	X	X	X	X	X	X	X
In map:	X	X	✓	X	X	✓	X	X
max =	0	0	2-0	2	2	5-0	5	5
			= 2			= 5		

<u>15:0</u>
13:1 48:7
7:3
8:4
25:6

Map

```

public int maxLen(int[] A, int n) {
    Map<Integer, Integer> mpp = new HashMap<>();
    int maxi = 0;
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += A[i];
        if (sum == 0) {
            maxi = i + 1;
        }
        else {
            if (mpp.containsKey(sum)) {
                maxi = Math.max(maxi, i - mpp.get(sum));
            }
            else {
                mpp.put(sum, i);
            }
        }
    }
    return maxi;
}

```

TC: $O(N)$

SC: $O(N)$

TC: $O(N)$

SC: $O(1)$